gdsddp: module reference

This document explains every module of the public implementation of good-deal bounds in SDDP (Berthet and Tveit, 2026), what each function or class does, what it takes in and what it returns, and how the pieces connect. Read it next to the code: the docstrings carry the same explanations in shorter form. Notation: T is the number of weekly stages (t = 0..T-1), K the number of lattice states per week, m a state, m' a successor state, delta the physical transition probabilities and q a distorted measure.

How the pieces fit together

Data enters through **data.py** (prices, inflow files) and **forwardcurve.py** (futures panel). The notebooks turn the data into the parameters of the two stochastic models in **models.py**. **lattice.py** simulates the models and quantises the paths into a Markov lattice. **contracts.py** puts the forward contracts on that lattice. **hydro.py** holds the plant and the production LP of one node, **gooddeal.py** the worst-case measure of one node, and **sddp.py** runs the SDDP loop that combines them into a policy (a set of cuts). **simulate.py** evaluates a policy on fresh paths and **plots.py** draws the figures.

```
data.py / forwardcurve.py  ->  notebooks 01-03  ->  parameters (JSON)
                                                          |
models.py  ----simulate_paths---->  lattice.py  ----->  Lattice
                                                          |
contracts.py (ContractSystem on the Lattice)              |
hydro.py     (Cascade, Economics, NodeLP)                 v
gooddeal.py  (worst_case_measure)   ---->   sddp.train  ---->  Policy (cuts)
                                                          |
                                          simulate.simulate  ---->  trajectories, revenue
                                                          |
                                                    plots.py
```

Unit conventions: prices in EUR/MWh, storage in Mm³, flows in Mm³/week, energy factors in MWh per Mm³, money in EUR after tax. Weeks of the year run 1..52 (ISO week clipped to 52). Delivery weeks of contracts are 1-based stage numbers; inside the code stages are 0-based.

models.py: the stochastic models

Two models generate the uncertainty: the spot price and the two reservoir inflows.

`week_of_year(t, start_week=1) -> int`

Calendar week (1..52) of stage t when stage 0 falls in calendar week start_week. Every seasonal lookup (price season, inflow parameters) goes through this function, so the horizon can start in any week.

`class PriceModel`

Schwartz and Smith (2000) two-factor model written additively in EUR/MWh and in weekly steps:

```
P_t   = xi_t + chi_t + s(w_t)
chi_t = phi_chi * chi_{t-1} + sigma_chi * eps_chi,t      short-term deviation, mean reverting
xi_t  = xi_{t-1}            + sigma_xi  * eps_xi,t       long-term level, random walk
s(w)  = A cos(2 pi (w - w0) / 52)                        deterministic season
```

field	meaning	source
phi_chi, sigma_chi, sigma_xi	dynamics of the two factors	notebook 3 (forward-curve PCA)
season_amplitude A, season_phase w0	the yearly cycle	notebook 1 (spot history)
xi0	long-term level at the start of the horizon	notebook 1 (recent deseasonalised mean)
chi0	short-term deviation at the start (default 0)	optional

- **season(weeks)**: s(w) for an array of weeks of the year.
- **simulate(n_paths, T, rng, start_week)**: returns (chi, xi, season, eps_total). chi and xi are (n_paths, T) arrays, season is (T,) and depends only on the calendar; eps_total is the standardised innovation of the total weekly price change, which the inflow model uses to correlate inflow with price. The spot price is max(0, chi + xi + season).

- **expected_price(chi_t, xi_t, week_t, k)**: E[P_{t+k} | chi_t, xi_t] = xi_t + phi^k chi_t + s(w_{t+k}), the model-implied forward price; used by the sample-data generator and useful for explaining the PCA.
- **to_json / from_json**: parameter persistence.

`class InflowModel`

One periodic AR(1) per reservoir on the log of the weekly inflow, with one parameter set per week of the year and correlated innovations between the reservoirs:

```
log I_{r,t} = mu_{r,w} + phi_{r,w} log I_{r,t-1} + sigma_{r,w} e_{r,t},   r in {upper, lower}
corr(e_upper,t, e_lower,t) = rho_w
```

field	shape	meaning
mu, phi, sigma	(2, 52)	intercept, persistence and innovation sd per reservoir (row 0 upper, row 1 lower) and week
rho	(52,)	correlation of the two reservoirs' innovations per week
log_mean	(2, 52)	mean log inflow per week; the simulation starts from it
n_years	int	years of data behind the fit

- **InflowModel.fit(weekly, rho_fixed=None)** (classmethod): ordinary least squares per week of the year on the pairs (log I_{t-1}, log I_t) of a DataFrame with columns upper, lower, woy. The residual correlation per week gives rho_w (overridden by rho_fixed). The fitted model carries a residuals DataFrame for diagnostics.
- **simulate(n_paths, T, rng, start_week, eps_price=None, rho_price=0)**: (n_paths, T, 2) inflows in Mm³/week. Each week draws a correlated pair of standard normals. With eps_price and rho_price both innovations receive the common component rho_price * eps_price and the pair's own correlation is adjusted so that the upper/lower correlation stays rho_w and every innovation keeps unit variance.
- **to_json / from_json**: parameter persistence.

`simulate_paths(price, inflow, n_paths, T, seed, rho_price_inflow, start_week) -> dict`

Joint simulation of both models with one random generator. Returns (n_paths, T) arrays under the keys price_chi, price_xi (= xi + season, so that price_chi + price_xi is the price), price (floored at 0), inflow_upper and inflow_lower. This dict is what build_lattice consumes.

lattice.py: quantisation into a Markov lattice

SDDP needs the uncertainty in a finite form. For every week one K-means (scikit-learn) over all coordinates jointly places K states where the simulated paths are; transition probabilities between the states of consecutive weeks are empirical frequencies. Clustering jointly, rather than one grid per coordinate, avoids states in corners that never occur and spends the K states where the probability mass is.

`class Lattice`

field	meaning
T, K	weeks and states per week
coords	dict name -> (T, K) centroid values in original units: price_chi, price_xi, inflow_upper, inflow_lower
transitions	list of T-1 dense (K, K) row-stochastic matrices
meta	n_paths, seed, coordinate names, mean share of variance left within clusters

- **price(t, m)**: received spot price at a state, the sum of the price coordinates floored at 0; **price_matrix()** is the same for all (t, m) at once.
- **inflow(t, m)**: (upper, lower) inflow at a state.

- **successors(t, m, threshold=1e-6)**: indices and renormalised probabilities delta of the successor states with probability above the threshold. Only these states enter the node problems, which keeps every node problem small.
- **marginals()**: (T, K) probability of each state when week 0 is uniform; used for strikes and the terminal value.
- **save(path) / Lattice.load(path)**: npz persistence.

`build_lattice(paths, K, seed=42, n_init=4, coordinates=(...), verbose=True) -> Lattice`

For each week t: (1) stack the chosen coordinates of all paths, taking logs of the inflows, and standardise each column; (2) KMeans(n_clusters=K, n_init=n_init, random_state=seed) on the standardised matrix; (3) centroids in original units as the mean of the members (exp of the mean log for inflows); (4) states sorted by received price. Then the transition matrix of each week pair is the count of path moves between labels, row-normalised; a state that no path visits gets the marginal distribution of the next week as its row. The default coordinates keep the two price factors separate; passing ("price", "inflow_upper", "inflow_lower") collapses the price to one coordinate.

contracts.py: forward contracts on the lattice

A forward contract delivers one MWh in every week of [start, end] and settles against the weekly spot price. The good-deal step needs the contract's fair value at every (t, m) and its cash-augmented increment to each successor.

`class Contract(id, start, end)`

Frozen dataclass; start and end are 1-based delivery weeks, length = end - start + 1.

`class ContractSystem(contracts, lattice)`

On construction it computes for every contract j:

```
F_j(t, m)  = E[ sum of remaining delivery-week prices | m at t ] / L_j   (backward recursion)
strike_j   = E[ average spot over delivery ]  under the lattice marginals (fair forward, zero premium)
dF_j(m,m') = F_j(t+1, m') - F_j(t, m) + P_t(m)/L_j * [t in delivery]           (cash-augmented increment)
```

- **active(t)**: ids of the contracts not yet fully delivered at week t: those with a martingale constraint at that week.
- **increments(t, m, succ_idx)**: (active ids, dF) with dF of shape (n_active, n_successors). Memoised, because neither F nor the successor support changes during SDDP.
- **check_martingale()**: largest |E_delta[dF]| over all nodes; should be of the order 1e-12. The fair forward has zero drift under the physical probabilities, and the good-deal step preserves that under q.
- fields: ids, P (the (T, K) price matrix), F (dict id -> (T+1, K)), strikes, L, start0/end0 (0-based delivery bounds).

hydro.py: the plant and the production LP

`Reservoir, Cascade, Economics`

object	fields	notes
Reservoir	name, s_max, s_init, x_max, energy_factor	usable storage, start storage, max weekly release, MWh per Mm³
Cascade	upper, lower; property s0	upper water released or spilled flows into the lower reservoir within the week
Economics	rf_annual, tau_C, tau_R; properties tau, beta	tau = tau_C + tau_R on revenue; beta = weekly after-tax discount factor

`gurobi_env() -> gp.Env`

One silent, single-threaded Gurobi environment shared by every model of the process.

`class NodeLP(cascade, econ, price, inflow, succ_idx, terminal_value=(0, 0))`

The production LP of one node (t, m), built once and kept in memory. Each solve only changes the incoming storage, the successor weights and the cuts added since the last solve.

```
max  (1 - tau) P (xi_U x_U + xi_L x_L) + beta * sum_m' w_m' alpha_m'   [+ beta * terminal . s' at t = T-1]
s.t. s'_U + x_U + sp_U              = s_U + I_U        (upper balance, dual = water value of upper storage)
     s'_L + x_L + sp_L - x_U - sp_U = s_L + I_L        (lower balance, dual = water value of lower storage)
     0 <= x <= x_max,  sp >= 0,  0 <= s' <= s_max
     alpha_m' <= theta_c + gU_c s'_U + gL_c s'_L      for every cut c of successor m'  (alpha_m' <= 0 until a cut ex
```

- **add_cuts(cuts_next)**: cuts_next maps successor state -> list of (theta, gU, gL); appends the cuts the model does not have yet.
- **solve(s_in, weights, cuts_next=None, cold=False) -> NodeSolution**: weights are delta (risk neutral) or q* (good deal). cold=True discards the warm-start basis so that identical inputs give identical vertices (used in simulation).
- **NodeSolution**: objective, release (2,), spill (2,), storage_out (2,), water_values (2,) = the duals of the balances.
- **dispose()**: frees the Gurobi model.

`terminal_water_values(lattice, cascade, econ) -> (termU, termL)`

EUR per Mm³ credited to the storage left at the end of the horizon: expected end-of-horizon after-tax price times the energy the water can still produce (upper water also passes the lower plant). Without it the policy would empty the reservoirs in the last weeks.

gooddeal.py: the worst-case measure of one node

A risk-neutral producer values next week's continuation value V(m') by its expectation under delta. The good-deal bound takes the lowest expectation over all measures q that (a) stay within a chi-squared distance of delta, sum q²/delta <= 1 + h_w² with the weekly Sharpe bound h_w = h / sqrt(52) (Cochrane and Saá-Requejo, 2000), and (b) keep every live forward a martingale, sum q dF_j = 0. The martingale rows mean the adversary cannot move probability along directions the forwards span: hedgeable price risk is priced by the market, not by the producer's aversion; the budget is spent on what the contracts cannot reach, mainly inflow.

`weekly_ball_radius(h) -> float`

Right-hand side of the chi-squared constraint, 1 + (h / sqrt(52))².

`worst_case_measure(V, delta, dF, h, flat_tol=1e-6) -> GoodDealSolution`

```
min_q  sum_m' q(m') V(m')
s.t.   sum q = 1,  q >= 0
       sum_m' q(m') dF_j(m') = 0            for every row j of dF
       sum_m' q(m')^2 / delta(m') <= 1 + h_w^2
```

Solved directly by Gurobi as a convex QCQP. Rows of dF with no variation across the successors are dropped. If h = 0, there is a single successor, or V is flat (nothing to distort) the function returns delta without calling the solver. GoodDealSolution holds q, value = sum q V, budget_used = (sum q²/delta - 1) / h_w² (1.0 when the ball binds), fell_back (True only if Gurobi failed, in which case q = delta) and a status string. q* replaces delta as the successor weights in NodeLP.solve, which is how the risk adjustment enters every cut.

sddp.py: the SDDP loop

The value of water V_t(m, s) is concave in the storage s, so every cut theta + g . s is an upper bound and the minimum over the cuts of a node is the approximation the LP uses. One iteration is a forward pass along one sampled path and a backward pass over every state of every week.

`class SDDPConfig`

field	default	meaning
iterations	30	maximum SDDP iterations
tol, window	1e-4, 10	stop when the bound moved less than tol (relative) over window iterations
seed	42	random generator of the forward passes

field	default	meaning
seed_levels	(0.1, 0.5, 0.9)	fractions of capacity for the initial backward sweeps
h	0.0	annual Sharpe bound; 0 = risk neutral
log_every	5	print cadence

`class Policy`

The trained policy: cuts[t][m] = list of (theta, gU, gL); history, a DataFrame with one row per iteration (bound, forward_value, fallbacks, seconds, n_cuts); config; name; q_star and budget_used, the worst-case measure and budget use of every node from the last backward pass (good-deal runs only). Property bound = the last first-stage bound. cuts_at(t) returns the cuts of week t in the map form NodeLP.add_cuts expects.

`train(lattice, cascade, econ, contracts, config, name, log=print) -> Policy`

- Precomputes the price matrix, the inflows, the successor supports and the terminal values; node LPs are built lazily and cached.
- **Seeding**: one backward sweep along each constant storage trajectory in seed_levels so every node has cuts over the storage range.
- **Forward pass**: start at a uniformly drawn state with the initial storage; at each week solve the node LP with the current cuts and delta weights, record the storage it leaves behind, move to a sampled successor; accumulate the discounted revenue of the path (forward_value).
- **Backward pass**: for t = T-1 down to 0 and every state m: if good deal and t < T-1, evaluate the cut envelope of week t+1 at the trial storage (V of each successor) and call worst_case_measure with the contract increments of the node; solve the node LP with q* (or delta) as weights; add the cut (objective - g . s_trial, gU, gL) where g are the water-value duals. The stage-0 objectives averaged over the states are the bound.
- **Convergence**: relative range of the bound over the window below tol, or the iteration cap; the last pass records q* and budget use for the figures. Everything is single-threaded.

`_envelope(cuts_t, s) -> (K,)`

Cut-implied value of every state of a week at storage s: the minimum over its cuts (0 where no cut exists).

Module constant NEG_PROB = 1e-6: successor probabilities below it are dropped from the supports.

simulate.py: evaluating a policy

`simulate(policy, lattice, cascade, econ, n_paths=500, seed=7, log=print) -> dict`

Simulates the policy under the PHYSICAL transition probabilities: the good-deal measure is a valuation device inside the cuts, not a forecast. Each path starts at a uniformly drawn state with the configured storage; week by week the node LP (with the trained cuts, delta weights and a cold solve) decides release and storage, and the next state is drawn from the lattice. All randomness is drawn up front so different policies see the same paths. Returns arrays state (N, T), price (N, T), inflow (N, T, 2), storage (N, T+1, 2), release, spill (N, T, 2), revenue (N, T) after tax, terminal (N,) and total (N,) = discounted sum of revenue plus the terminal credit.

`revenue_summary(total, alpha=0.05) -> dict`

mean, std, p5, cvar5 (mean of the worst alpha share), p95, n_paths.

plots.py: figures

function	shows
style()	matplotlib defaults used by the notebooks
lattice_fan(lattice, coord, title, ax, unit)	all K centroids of one coordinate over the weeks
lattice_scatter(paths, lattice, t, x, y, ax)	the simulated paths of week t in the (x, y) plane with the lattice states on top

function	shows
convergence(policies, ax)	the first-stage bound per iteration for several policies
storage_fan(sims, which, label, ax)	median and 10-90 % band of a reservoir's storage per policy
revenue_hist(sims, ax)	histogram of total revenue per policy
bound_vs_h(table, ax)	policy value against the Sharpe bound h
distortion_tilt(policy, lattice, ax)	E_q - E_delta of next week's inflow and price, in standard deviations, per week

Colours: a light-to-dark blue ramp (RAMP) for ordered families, INK for the reference series, GREEN reserved for a good-deal series in two-series figures.

data.py: data access

function	purpose
fetch_system_price(start, end, cache, timeout)	hourly Nord Pool system price from Energi Data Service (dataset Elspotprices, PriceArea SYSTEM), one request per year, honours the API's Retry-After header, caches to CSV
load_price_file(path)	a CSV with a date column and one price column (hourly or daily) as a Series
weekly_mean(series)	weekly means with columns price, woy; partial edge weeks dropped
fit_season(weekly, start, end)	least-squares cosine fit on a window: season_amplitude, season_phase, level, residual
load_station_flows(path)	station flows in m³/s at any regular time step -> weekly Mm³ of the upper and lower reservoir (upper, lower, woy)
deseasonalised_ar1_residuals(series, woy)	residuals of a per-week-of-year AR(1); used for the price-inflow innovation correlation
save_json / load_json	parameter files in data/processed/

Constants: ROOT, RAW (data/raw, git-ignored), SAMPLE (data/sample), PROCESSED (data/processed, git-ignored), UPPER_STATIONS = Namsvatn, Vekteren, Limingen; LOWER_STATIONS = Tunnsjø, Tunnsjøflyene.

forwardcurve.py: forward curve and PCA

Turns a futures panel into the three dynamic parameters of PriceModel.

function	purpose
parse_series(code)	delivery window and type (W, M, Q, Y) from a Nasdaq-style contract code such as ENOW05-16, ENOMSEP-17, ENOQ1-18, ENOYR-20
load_panel(path)	the panel with parsed windows; keeps base-load futures, drops options, one quote per (day, type, window)
build_block_curve(panel, k_max=104)	BLOCK curve per trading day: for week k ahead the shortest contract type that covers it, day-weighted across a boundary; returns (prices, ctypes)
build_smooth_curve(panel, k_max=104, smoothness=1.0, grid_extra=60)	SMOOTH curve per trading day: the weekly curve with the least squared second differences that reprices every quoted contract as the day-weighted average over its delivery weeks (penalised least squares, one linear solve per day); the default in notebook 3
repricing_error(panel, prices)	RMSE between quoted contract prices and the curve's average over their delivery weeks, per contract type
weekly_fixed_delivery_returns(prices, ctypes)	curve sampled on the last trading day of each ISO week; r[t, k] = log F_t(k) - log F_{t-1}(k + d); cells whose contract type changed are masked
pca(R, min_obs=100, max_row_missing=0.2)	scikit-learn PCA on the covariance of the returns (sparse maturities dropped, gaps mean-filled): k, cov, eigenvalues, share, loadings (first three, PC1 positive at the front), vol, scores
model_cov(theta, k)	the two-factor covariance sigma_xi² + sigma_chi² phi^(k_i + k_j); theta = (log sigma_xi, log sigma_chi, logit phi)
fit_two_factor(S, k)	weighted least squares of model_cov on the empirical covariance: sigma_xi, sigma_chi (log units per week), phi_chi, half-life, trace share, vol RMSE

The log-unit volatilities are multiplied by a reference price level (the mean curve level of the window) to obtain the EUR/MWh parameters of the additive price model.

experiment.py: the run as three steps

Notebook 4 as reusable functions so that a large case can be split into independent jobs. All steps read and write results/<name>/.

function	purpose
load_parameters(processed)	PriceModel, InflowModel, rho_price_inflow and their sources: data/processed/ when notebooks 1-3 have run, literature values otherwise
default_cascade(), default_contracts(T), parse_contracts(spec, T)	the thesis plant; contracts by horizon length (Q2, H2, and Y2 when T > 52) or from a string such as 'H2:27-52,Y2:53-104'
build(name, T, K, n_paths, seed, start_week, n_init)	simulate the models and build the lattice; writes lattice.npz and config.json
train_and_simulate(name, h, iterations, tol, window, sim_paths, seed, contracts)	train one policy on the saved lattice and simulate it; writes /history.csv, cuts.npz, sim.npz, q_star.npz, summary.json
report(name)	collect the finished policies into summary.csv and the standard figures (convergence, storage and revenue, bound against h, distortion tilt)

scripts/run_experiment.py is the command-line front end (build / train / report); the train steps are independent and can run as separate jobs.

scripts, tests and notebooks

- **scripts/make_sample_data.py**: writes the synthetic sample (daily system price, daily station inflows, a daily futures panel) from the models with round parameters; nothing in it comes from the real data.
- **tests/test_smoke.py**: end-to-end check on the sample (inflow fit, lattice, martingale, one good-deal node, a short RN and GD training, simulation) and a PCA parameter-recovery test.
- **notebooks/01** price data from the API, season and starting level; **02** inflow analysis and InflowModel.fit; **03** forward curve, PCA and the two-factor fit; **04** parameters at the top, then lattice, contracts, training, simulation, figures and results/.